Kubernetes Resource Management – Troubleshooting Guide

1. REQUESTS & LIMITS ISSUES

Scenario 1: Pod Stuck in Pending

Problem:
Pod remains in Pending state.

Root Cause:
Node does not have enough resources to satisfy the pod's **requests**.

Diagnosis:

kubectl describe pod <pod-name>

Look for:

Insufficient cpu / memory

Solution:

- Reduce resource requests
- Add more nodes
- Enable cluster autoscaling

Scenario 2: Node Overloaded

Problem:
All pods are running, but node performance is poor.

Root Cause:
Requests are not defined → scheduler over-allocates pods.

Solution:
Define resource requests in all pods:

requests:

cpu:

memory:

Scenario 3: Application Slowness

Problem:
Application becomes slow under load.

Root Cause:
CPU throttling due to low CPU limit.

Solution:

- Increase CPU limits
- Remove strict CPU limits (if appropriate)

Scenario 4: OOMKilled Errors

Problem:
Containers terminate with:

Reason: OOMKilled

Root Cause:
Memory limit too low.

Solution:

- Increase memory limit
- Monitor usage:

kubectl top pod

Scenario 5: CrashLoopBackOff

Problem:
Pod enters CrashLoopBackOff.

Root Cause:
Repeated OOMKills or application crash.

Solution:

- Increase memory limits
- Fix application memory usage

---

2. QoS CLASS ISSUES

Scenario 6: BestEffort Pods Getting Killed

Problem:
Certain pods are always terminated first.

Root Cause:
Pods are in BestEffort QoS (no requests/limits defined).

Solution:
Define both requests and limits.

Scenario 7: Guaranteed Pod Also Terminated

Problem:
Guaranteed pod gets killed unexpectedly.

Root Cause:

- Node completely out of memory
- System-level pressure

Solution:

- Increase node capacity
- Reduce cluster overcommitment

Scenario 8: Burstable Pods Killed Early

Problem:
Burstable pods terminated sooner than expected.

Root Cause:
Memory usage exceeds request significantly.

Solution:
Increase memory requests.

Scenario 9: Unknown QoS Class

Diagnosis:

kubectl get pod <pod> -o jsonpath='{.status.qosClass}'

---

3. LIMITRANGE ISSUES

Scenario 10: Default Resources Automatically Assigned

Problem:
Resources appear even though not defined.

Root Cause:
LimitRange is applied in namespace.

Diagnosis:

kubectl get limitrange -n <namespace>

Scenario 11: Pod Creation Fails (Limit Exceeded)

Error:

must be less than max

Root Cause:
LimitRange maximum exceeded.

Solution:

- Adjust pod resource values
- Modify LimitRange

Scenario 12: Unexpected Default Values

Root Cause:
LimitRange defaultRequest/default values applied.

Diagnosis:

kubectl describe limitrange

---

4. RESOURCEQUOTA ISSUES

Scenario 13: Quota Exceeded Error

Error:

exceeded quota

Root Cause:
Namespace resource limits reached.

Diagnosis:

kubectl describe resourcequota

Solution:

- Delete unused resources
- Increase quota

Scenario 14: Deployment Not Scaling

Problem:
Increasing replicas fails.

Root Cause:
Quota limit reached.

Solution:
Increase:

- CPU quota
- Memory quota
- Pod count

Scenario 15: Requests Quota Exceeded

Problem:
Requests quota exceeded but limits available.

Root Cause:
Requests and limits tracked separately.

Solution:

- Increase request quota
- Reduce requests

---

5. EVICTION ISSUES

Scenario 16: Pods Suddenly Deleted

Problem:
Pods disappear unexpectedly.

Root Cause:
Node memory pressure.

Diagnosis:

kubectl describe node <node>

Look for:

MemoryPressure: true

Scenario 17: BestEffort Pods Always Removed First

Root Cause:
Eviction order:

BestEffort → Burstable → Guaranteed

Solution:
Avoid BestEffort pods in production.

Scenario 18: Pod Recreated Automatically

Problem:
Pod disappears and new one appears.

Root Cause:
Eviction + controller recreation (Deployment/ReplicaSet).

Solution:
Expected behavior.

Scenario 19: Disk Pressure Eviction

Diagnosis:

DiskPressure: true

Root Cause:
Node disk is full.

Solution:

- Clean logs
- Increase disk size
- Use persistent volumes

---

6. OOMKILL ISSUES

Scenario 20: Continuous Container Restart

Problem:
Container repeatedly restarts.

Root Cause:
Memory limit too low.

Solution:

- Increase memory limit
- Optimize application memory

Scenario 21: OOMKilled Below Limit

Problem:
Container killed even below limit.

Root Cause:
Node-level OOM (not container limit breach).

Solution:

- Check node memory usage
- Reduce cluster load

Scenario 22: Java/Node Apps OOM

Root Cause:
Runtime not container-aware.

Solution Example (Java):

-XX:+UseContainerSupport

---

7. ADVANCED SCENARIOS

Scenario 23: Overcommitment Instability

Problem:
Cluster crashes under load.

Root Cause:
Total limits far exceed node capacity.

Solution:

- Control request/limit ratios
- Avoid aggressive overcommitment

Scenario 24: Uneven Pod Distribution

Problem:
One node overloaded, others idle.

Root Cause:
Improper requests or scheduling imbalance.

Solution:

- Define accurate requests
- Use pod anti-affinity

Scenario 25: High Latency Under Load

Root Cause:
CPU throttling.

Solution:

- Increase CPU limits
- Tune autoscaling

Scenario 26: Pods Evicted During Scaling

Root Cause:
Autoscaler delay causes temporary pressure.

Solution:

- Tune autoscaler parameters
- Maintain buffer capacity

🔧 DEBUGGING COMMANDS

Pod-Level Debugging

kubectl describe pod <pod>

kubectl logs <pod>

Node-Level Debugging

kubectl describe node <node>

Check:

- MemoryPressure
- DiskPressure

Resource Usage

kubectl top pod

kubectl top node

QoS Check

kubectl get pod -o jsonpath='{.status.qosClass}'

Quota & Limits

kubectl describe resourcequota

kubectl describe limitrange

TROUBLESHOOTING FLOW

Use this order while debugging:

1. Pod Pending → Requests issue

2. Container restarting → OOMKill

3. Pod deleted → Eviction

4. Deployment blocked → ResourceQuota

5. Defaults applied → LimitRange

6. Priority issue → QoS class
